TRICODE DATABASE DOCUMENTATION

PROJECT Members

- Mwizerwa Keza Megane
- Nganji Jospin
- Lea ABATONI

1. TABLE OF CONTENT

- Introduction
- System Overview
- Entity Relationship Diagram (ERD)
- Design Rationale and Justification
- Database Schema Description
- Data Dictionary
- Sample Queries and Results
- Security and Data Integrity Rules
- Conclusion

Introduction & System Overview

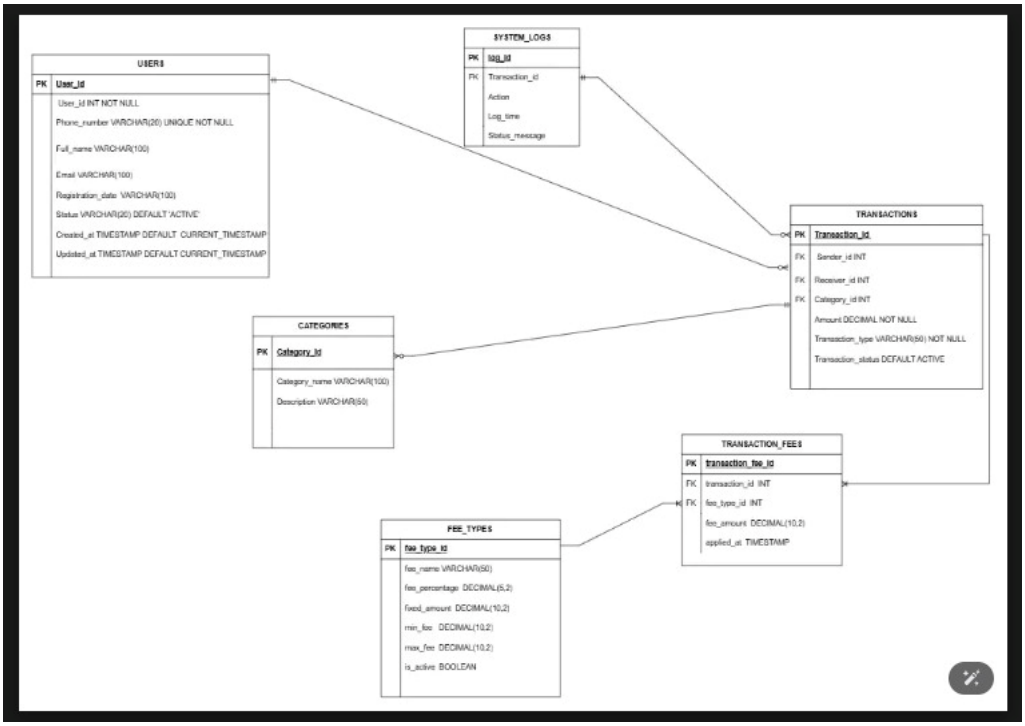
This database AMAFARANGADB is designed to store and manage MOMO sms transaction details.

This system is designed to track and store a user's transaction through efficient querying design and accurate reporting of those financial transactions overtime.

Here is a short description of how data flows in our system:

SMS → parsed → stored → queried

ERD Diagram and FULL Documentation



USERS

This particular entity stores all the information about the user, their name, phone number, registration date and the status of their account. With this particular entity we are able to trace the relationship with other tables and track the particular user's transactions.

TRANSACTIONS

With this entity we are able to clearly see what the user did due to the FKs that referenced the USERS.User_id to know whether this particular user was a sender or a receiver.

We have foreign keys like: Sender_id and Receiver_id that help us track more efficiently what the user did more clearly and be able to categorize them well.

From the picture above the relationship between USERS table and TRANSACTIONS table is a one to many relationships where by one user can have zero to many transactions whereas every transaction must have exactly one sender and exactly one receiver.

CATEGORIES

With this entity we are able to put into categories the different transactions that were made such as : withdrawals, transfers , payments or utility bills.

This helps a lot in further accuracy of the systems in determining the charge fees for each transactions and the different Actors involved that initiated the transaction.

FEE_TYPES

The fees type table defines how transaction fees are calculated, including the percentage based fees and the fixed fees, minimum and maximum limits.

TRANSACTION_FEES

This Entity is what we call a JUNCTION TABLE, this is what resolves the many-to-many relationship between transactions and fee types.

Meaning that a single transaction can have multiple fees applied and the same fee type can apply to many transactions.

SYSTEM_LOGS

The systems log table does not store money or records of the users, instead it tracks what happened in the system when a particular transaction took place that is why it has transaction_id as its Foreign key.

Design Rationale and Justification

The design of this database follow the 3NF Normal Form, where by each table stores one type of information and reliant of their respective primary keys.
For example: We have the USERS table, separate from the TRANSACTION table, separate from TRANSACTION FEES table etc.
While designing this database, we put into consideration the relationship between the entities and hence introduced the concept of Foreign keys.
For a little bit of more context take an example between SYSTEM LOGS table and TRANSACTION table,
The System logs tables has Foreign key as transaction_id in order to create the relationship where we are able to track what happened in the system when a particular transaction took place.
Another one of design decision would be the TRANSACTION FEES table, since we are working with 3NF Normal, each table stores a particular type of data, but in addition to that TRANSACTION FEES table is a JUNCTION Table that helps establish the connection of a many-to-many relationship and in this case the relationship between TRANSACTIONS table and FEE TYPES table.
Most of other tables have relationships amongst themselves with the help of Foreign keys but it is also very important to be able to trace how the transactions are being processed in the system in order to able to know whether for example it was a successful payment or completed, etc.

DATA DICTIONARY

- **USERS:** Stores registered user information

```sql
use tricode_db;
CREATE TABLE USERS (
    User_id INT PRIMARY KEY AUTO_INCREMENT,
    Phone_number VARCHAR(20) UNIQUE NOT NULL,
    Full_name VARCHAR(100),
    Email VARCHAR(100),
    Registration_date VARCHAR(100),
    Status VARCHAR(20) DEFAULT 'ACTIVE',
    Created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

- **TRANSACTIONS :** Stores the financial records of the users

```sql
CREATE TABLE TRANSACTIONS (
    Transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    Sender_id INT NOT NULL,
    Receiver_id INT NOT NULL,
    Category_id INT NOT NULL,
    Amount DECIMAL(15,2) NOT NULL,
    Transaction_type TIMESTAMP NOT NULL,
    Transaction_status VARCHAR(20) DEFAULT 'ACTIVE',

    FOREIGN KEY (Sender_id) REFERENCES USERS(User_id),
    FOREIGN KEY (Receiver_id) REFERENCES USERS(User_id),
    FOREIGN KEY (Category_id) REFERENCES CATEGORIES(Category_id)
);
```

- **CATEGORIES:** Defines the different transaction categories

```sql
CREATE TABLE CATEGORIES (
    Category_id INT PRIMARY KEY AUTO_INCREMENT,
    Category_name VARCHAR(100),
    Description TEXT
);
```

- **FEE_TYPES:** This entity helps define the fee calculation rules

```sql
);
CREATE TABLE FEE_TYPES (
    fee_type_id INT PRIMARY KEY AUTO_INCREMENT,
    type_name VARCHAR(50),
    fee_percentage DECIMAL(5,2) DEFAULT 0.21,
    fixed_amount DECIMAL(15,2),
    min_fee DECIMAL(10,2),
    max_fee DECIMAL(10,2),
    is_active BOOLEAN DEFAULT TRUE
);
```

- **<u>TRANSACTION_FEES:</u>** This entity stores the applied fees per transaction

```sql
CREATE TABLE TRANSACTION_FEES (
    transaction_fee_id INT PRIMARY KEY AUTO_INCREMENT,
    transaction_id INT NOT NULL,
    fee_type_id INT NOT NULL,
    fee_amount DECIMAL(10,2),
    applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (transaction_id) REFERENCES TRANSACTIONS(Transaction_id),
    FOREIGN KEY (fee_type_id) REFERENCES FEE_TYPES(fee_type_id)
);
```

- **<u>SYSTEM_LOGS</u>:** Stores system activities per transaction

console | FEE_TYPES | SYSTEM_LOGS | TRANSACTION_FEES | TRANSACTIONS | USER_TRANSACTIONS | CATEGORIES

Tx: Auto | Playground | tricode_db

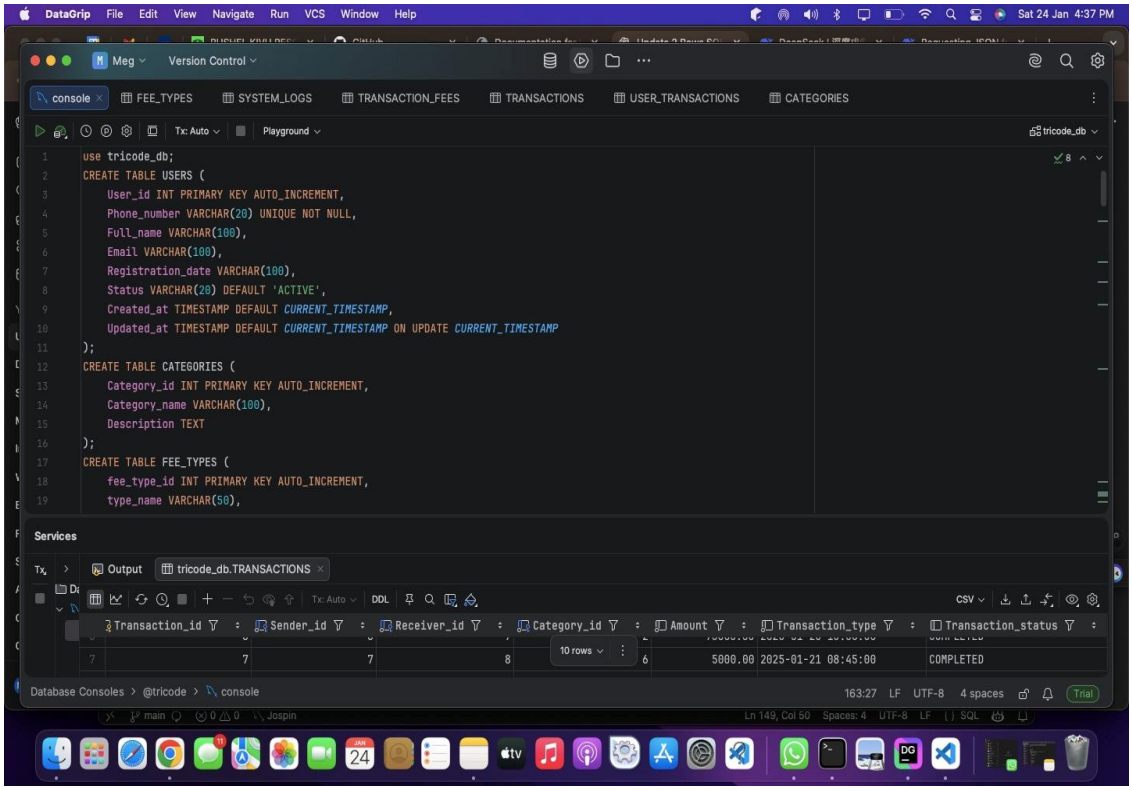
```sql
CREATE TABLE SYSTEM_LOGS (
    Actor VARCHAR(100),
    Log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Status_message TEXT,

    FOREIGN KEY (transaction_id) REFERENCES TRANSACTIONS(Transaction_id)
);
```

<u>CRUD OPERATIONS</u>

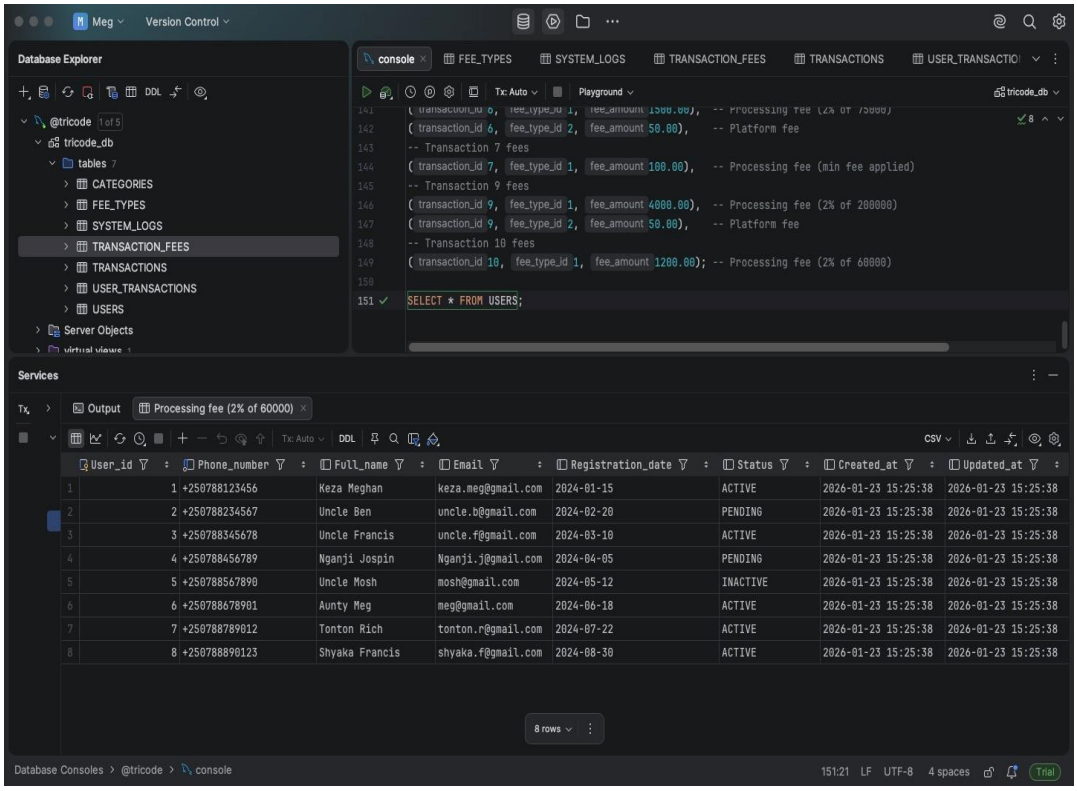
<u>CREATE</u>

CREATE TABLE USERS (User_id INT NOT NULL PRIMARY KEY, Phone_number VARCHAR(20) UNIQUE NOT NULL, Full_name VARCHAR(100), Email VARCHAR(100), Registration_date VARCHAR(100), Status VARCHAR DEFAULT ‘ACTIVE’, Created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, Updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP);



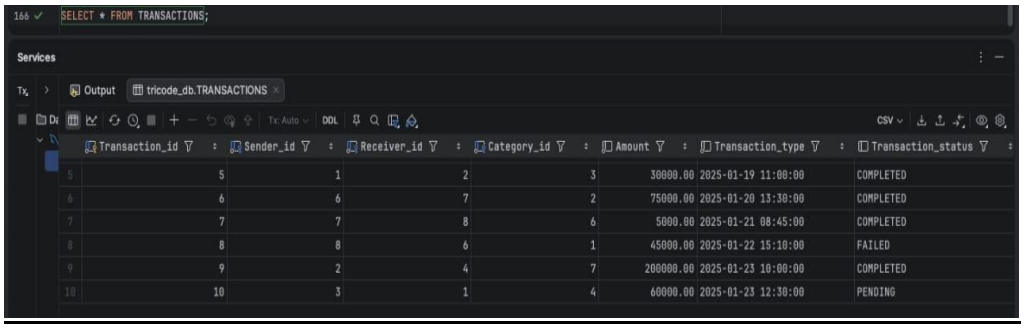
SELECT

SELECT * FROM USERS;



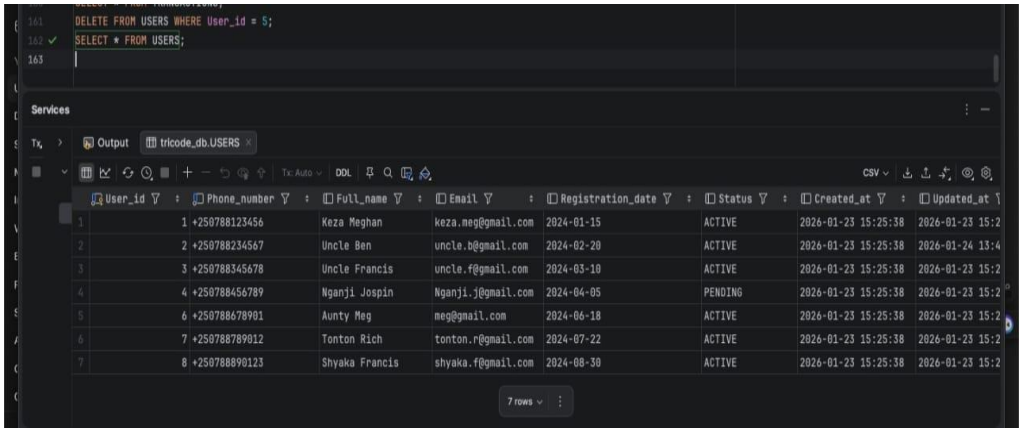
UPDATE

UPDATE TRANSACTIONS SET Reciever_id = 2 WHERE Transaction_id = 5;



DELETE

DELETE FROM USERS WHERE User_id = 5;



CONCLUSION

The purpose of this database is to store and track financial transaction in an orderly and structured way.
It records user information, tracking their financial details, categories, the different fees associated with the transactions and the status of the transactions in the system logs.
It keeps its data integrity with the use PRIMARY KEYS, FOREIGN KEYS and junction tables to establish clear and accurate relationships.

Prepared by TRICODE Team
On January 25, 2026